

Desenvolvimento de um Analisador Sintático SLR Modular com Verificação Semântica, Geração de Código de Três Endereços e Otimização

Title: Development of a Modular SLR Syntactic Analyzer with Semantic Verification, Three-Address Code Generation and Optimization

Jeffley Garçon¹, Ashley Saint Louis¹

¹Ciência da Computação – Universidade Federal da Fronteira Sul (UFFS)
Campus Chapecó – SC – Brasil

{jeffley, ashley}@estudante.uffs.edu.br

Abstract. Introduction: The syntactic analyzer is a core stage of a compiler, grouping tokens into structural hierarchies. **Objective:** This work implements a robust SLR parser engine for a hypothetical programming language, integrating frontend validation and intermediate code optimization. **Methodology:** The architectural core was built in Python, processing an input token stream using an explicit Shift-Reduce state table generated via context-free grammar transformations. It incorporates symbol table validation for scope checks, non-terminal syntax-directed translation into Three-Address Code (TAC), and basic block optimization. **Results:** The compiler successfully evaluated expressions, handled error mitigation, established a concrete symbol layout, and applied constant folding directly during compilation passes.

Keywords Compilers, SLR Parsing, Shift-Reduce, Syntax-Directed Translation, Three-Address Code, Constant Folding.

Resumo. Introdução: O analisador sintático é uma fase central de um compilador, agrupando tokens em hierarquias estruturais. **Objetivo:** Este trabalho implementa um motor de parsing SLR para uma linguagem de programação hipotética, integrando validação de frontend e otimização de código intermediário. **Metodologia:** O núcleo arquitetural foi construído em Python, processando um fluxo de tokens por meio de uma tabela explícita de estados Shift-Reduce gerada via transformações de gramáticas livres de contexto. Incorpora validação por tabela de símbolos para checagem de escopo, tradução dirigida por sintaxe de não-terminais para Código de Três Endereços (TAC) e otimização de blocos básicos. **Resultados:** O compilador avaliou expressões com sucesso, mitigou erros sintáticos, estabeleceu um layout concreto de símbolos e aplicou dobradura de constantes diretamente durante os passos de compilação.

Palavras-Chave Compiladores, Parsing SLR, Shift-Reduce, Tradução Dirigida por Sintaxe, Código de Três Endereços, Dobradura de Constantes.

1. Introdução

O desenvolvimento de compiladores exige a decomposição sistemática de fluxos textuais brutos em representações semânticas e operacionais acionáveis pela máquina alvo. Enquanto a análise léxica foca na individualização de categorias de *lexemas*, a análise

sintática estabelece as relações gramaticais estruturais que governam a corretude de um programa fonte.

Este projeto descreve o projeto detalhado e a implementação de um reconhecedor sintático SLR (Simple LR) acoplado a um motor semântico e a um gerador de código intermediário para uma linguagem hipotética imperativa estruturada. A engenharia do software foi fundamentada em Python, visando atender aos quatro eixos de avaliação propostos na disciplina de Construção de Compiladores: estruturação de Gramática Livre de Contexto (GLC) desambiguada, análise semântica descrita por Esquemas de Tradução, tradução para Código de Três Endereços (TAC) e otimização por dobradura de constantes.

2. Referencial Teórico e Engenharia da Gramática

Para viabilizar a construção de uma tabela de *parsing* determinística descendente com comportamento Bottom-Up (Shift-Reduce), o arcabouço sintático deve ser rigorosamente formalizado livres de ambiguidades, recursões à esquerda nocivas e fatores comuns não isolados.

2.1. Definição da Gramática Livre de Contexto (GLC)

A linguagem especificada introduz construções de controle imperativo (*se, enquanto*), declarações tipadas, atribuições e expressões matemáticas compostas por operadores aritméticos clássicos de diferentes precedências. A GLC final foi estruturada conforme as produções abaixo, onde símbolos em letras minúsculas e caracteres especiais representam os terminais (tokens):

- (0) $E' \rightarrow \text{PROG}$
- (1) $\text{PROG} \rightarrow \text{Bloco}$
- (2) $\text{Bloco} \rightarrow \{ \text{Comandos} \}$
- (3) $\text{Comandos} \rightarrow \text{Comando Comandos} \mid (4) \epsilon$
- (5) $\text{Comando} \rightarrow \text{se (Exp) Bloco senao Bloco}$
- (6) $\text{Comando} \rightarrow \text{enquanto (Exp) Bloco}$
- (7) $\text{Comando} \rightarrow \text{id} = \text{Exp} ;$
- (8) $\text{Comando} \rightarrow \text{tipo id} ;$
- (9) $\text{Exp} \rightarrow E$
- (10) $E \rightarrow E + T \mid (11) E - T \mid (12) T$
- (13) $T \rightarrow T * F \mid (14) T / F \mid (15) T$
- (16) $F \rightarrow (E) \mid (17) \text{id} \mid (18) \text{num}$

2.2. Cálculo de FIRST, FOLLOW e Itens Válidos

A geração da tabela de parsing SLR requer o mapeamento da coleção de itens LR(0), determinando as funções de fechamento (*Closure*) e transição (*Goto*) para estabelecer os estados do autômatos. Os conjuntos de direcionamento sintático calculados para a gramática foram indexados da seguinte forma:

Tabela 1. Conjuntos FIRST e FOLLOW da Linguagem Hipotética

Não-Terminal	FIRST	FOLLOW
E'	{ {, id, tipo, se, enquanto }	{ \$ }
PROG	{ { }	{ \$ }
Bloco	{ { }	{ \$, }, se, enquanto, id, tipo, senao }
Comandos	{ se, enquanto, id, tipo, ϵ }	{ } }
Comando	{ se, enquanto, id, tipo }	{ se, enquanto, id, tipo, } }
Exp, E , T , F	{ (, id, num }	{ ;,), +, -, *, / }

A resolução de conflitos de empilhamento e redução (Shift/Reduce) foi mitigada utilizando as restrições impostas pelo conjunto *FOLLOW* de cada não-terminal, mapeando as ações de redução estritamente quando o token subsequente na fita de entrada fizesse parte dos terminais permitidos pós-produção.

3. Arquitetura Modular da Implementação

Seguindo preceitos de manutenibilidade e coesão de software, a arquitetura foi desacoplada em quatro componentes core escritos em Python, permitindo o isolamento de cada etapa física descrita no projeto de compiladores.

3.1. Módulo estático de configuração: `config.py`

Concentra o mapeamento abstrato de todas as regras numeradas da GLC (0 a 18) e armazena o dicionário multidimensional que representa a matriz de parsing SLR de estados (Estados 0 a 44). Cada entrada mapeia um par (*Estado*, *Token*) para uma ação atômica, onde a string "S" denota *Shift* (empilhamento com destino ao próximo estado), "R" especifica um *Reduce* por regra, e "ACC" define o ponto de aceitação do programa fonte.

3.2. Motor de Parsing: `parser_slr.py`

Provê o loop de controle que simula o funcionamento do autômato de pilha Bottom-Up. Mantém duas estruturas de dados dinâmicas paralelas: uma pilha de controle de estados numéricos e uma pilha de valores de atributos sintáticos e semânticos. O motor consome sequencialmente a fita de saída originada pelo analisador léxico, alterando o estado interno do compilador através do mapeamento direto das ações contidas na tabela estática.

3.3. Esquema de Tradução Dirigida por Sintaxe: `suporte_compilador.py`

Engloba as rotinas semânticas executadas como ganchos imediatos a cada redução sintática confirmada pelo parser. Gerencia os erros de tipos, o escopo de identificadores e o barramento de emissão do código intermediário. Contém adicionalmente a função responsável pela varredura e otimização dos blocos lógicos.

3.4. Orquestrador Central: `app.py`

Atua como ponto de entrada da aplicação, inicializando fluxos de testes estruturados através da simulação de fitas lógicas e disparando o processo global de compilação.

4. Análise Semântica e Geração de Código Intermediário

A conformidade estrutural e a geração de código foram acopladas diretamente à execução das regras gramaticais através de ações semânticas associadas (SDT - *Syntax-Directed Translation*).

4.1. Tabela de Símbolos e Verificação Semântica (Etapa 2)

Toda vez que a regra 8 (Comando $\rightarrow$ tipo id ;) é reduzida, o identificador correspondente é capturado e inserido em uma tabela de símbolos baseada em dicionário indexado. O sistema valida em tempo de execução:

- **Erro de Dupla Declaração:** Se o identificador já existir no escopo atual da tabela de símbolos, uma mensagem de erro semântico impede a compilação.
- **Variáveis Não Declaradas:** Ao reduzir as produções 7, 17 e 18, o compilador inspeciona a tabela para garantir que o identificador alvo passou previamente por um comando de alocação de tipo. Caso contrário, emite falha semântica imediata.

4.2. Emissão de Código de Três Endereços - TAC (Etapa 3)

Para a síntese de código intermediário, cada nó não-terminal associado a expressões aritméticas propaga um atributo sintetizado denominado *lugar*, mapeando referências de memória ou constantes. Durante as reduções das regras matemáticas (10, 11, 13, 14), o sistema invoca uma rotina de geração de variáveis temporárias sob demanda ($t_1, t_2, \dots$), gravando instruções TAC na pilha global de representação intermediária. A Listagem 1 expõe a lógica de implementação unificada dessas ações semânticas.

Listing 1. Trecho do Esquema de Tradução Semântica e TAC em Python

```
1 blackdef acao_semantica(self, regra_num, nos_reduzidos):
2     blackif regra_num == 8: black black#black blackComando
3         black black->black blacktipoblack blackidblack black;
4         tipo, id_nome = nos_reduzidos[0][black'blackvalor
5             black'], nos_reduzidos[1][black'blackvalorblack']
6         blackif id_nome blackin self.tabela_simbolos:
7             blackraise SemanticError(fblack"blackVariavel
8                 black black'{blackid_nomeblack}'black blackja
9                 black blackdeclaradablack.black")
10        self.tabela_simbolos[id_nome] = {black'blacktipoblack'
11            : tipo}
12
13    blackelif regra_num == 7: black black#black blackComando
14        black black->black blackidblack black=black blackExp
15        black black;
16        id_nome, exp_lugar = nos_reduzidos[0][black'blackvalor
17            black'], nos_reduzidos[2][black'blacklugarblack']
18        blackif id_nome blacknot blackin self.tabela_simbolos:
19            blackraise SemanticError(fblack"blackVariavel
20                black black'{blackid_nomeblack}'black blacknao
21                black blackdeclaradablack.black")
22        self.codigo_intermediario.append(fblack"black{
23            blackid_nomeblack}black black=black black{
24            blackexp_lugarblack}black")
```

```

13
14     blackelif regra_num blackin [10, 11, 13, 14]: black black#
        black blackOperacoesblack black+,black black-,black
        black*,black black/
15         op1, operador, op2 = nos_reduzidos[0][black'blacklugar
            black'], nos_reduzidos[1][black'blackvalorblack'],
            nos_reduzidos[2][black'blacklugarblack']
16         temp = self.novo_temporario()
17         self.codigo_intermediario.append(fbblack"black{
            blacktempblack}black black=black black{blackop1
            black}black black{blackoperadorblack}black black{
            blackop2black}black")
18     blackreturn {black'blacklugarblack': temp}

```

5. Otimização de Código Intermediário (Etapa 4)

O módulo de otimização realiza uma análise linear sobre o vetor de código intermediário gerado, aplicando duas estratégias interdependentes: a **Propagação de Constantes** e a **Dobradura de Constantes** (*Constant Folding*).

O algoritmo varre sequencialmente o bloco de instruções TAC. Ao interceptar atribuições de valores estáticos para variáveis temporárias ou identificadores, o sistema armazena o par literal em um mapa local de rastreamento. Caso as instruções seguintes utilizem esses operandos mapeados, o otimizador substitui as referências nominais diretamente pelos valores numéricos literais correspondentes. Quando uma instrução contendo operadores aritméticos clássicos possui ambos os lados da igualdade preenchidos por constantes numéricas puras após a propagação, o compilador calcula o resultado final estaticamente em tempo de compilação, eliminando a instrução redundante e emitindo apenas o valor consolidado.

6. Resultados e Validação por Estudo de Caso

Para validar o comportamento integrado de todas as etapas lógicas do compilador, utilizou-se como estudo de caso uma fita de entrada gerada a partir da seguinte estrutura sintática contendo declaração e atribuição aritmética composta:

```
{ int x ; x = 10 + 5 ; }
```

O fluxo de processamento de estados executado pelo motor Shift-Reduce SLR transcorreu de forma bem-sucedida, registrando ordenadamente as ações de empilhamento de delimitadores e palavras reservadas e as reduções de expressões. A Tabela 2 demonstra as etapas finais impressas pelo sistema.

Após a consolidação sintática e aceitação da cadeia, o módulo de otimização processou a pilha TAC da Etapa 3. A seguir, realiza-se o confronto direto entre a saída intermediária padrão e o código otimizado gerado pela Etapa 4:

Tabela 2. Rastreamento de Execução do Parser SLR

Passo	Componente Acionado	Mensagem / Ação Emitida
1	Módulo Sintático	Reduziu pela Regra: $F \rightarrow \text{num}$
2	Módulo Sintático	Reduziu pela Regra: $T \rightarrow F$
3	Módulo Sintático	Reduziu pela Regra: $E \rightarrow T$
4	Módulo Sintático	Reduziu pela Regra: $F \rightarrow \text{num}$
5	Módulo Semântico	[Semântico] Declarado com sucesso: $x(\text{int})$
6	Geração de Código (TAC)	Emissão Intermediária: $t1 = 10 + 5$
7	Geração de Código (TAC)	Emissão Intermediária: $x = t1$
8	Módulo Sintático	[+] ACEITO: Cadeia reconhecida com sucesso!

Listing 2. Código Intermediário Bruto

```

1 // Saida TAC - Etapa 3
2 t1 = 10 + 5
3 x = t1

```

Listing 3. Código Intermediário Otimizado

```

1 // Saida TAC Otimizada -
  Etapa 4
2 t1 = 15
3 x = 15

```

O resultado final evidencia que o algoritmo detectou a operação aritmética composta exclusivamente por constantes na linha 1 ($10 + 5$) e realizou o *folding* para o valor consolidado 15, propagando o ganho subsequente para a instrução de atribuição do identificador x .

7. Conclusões

A construção modular do analisador sintático SLR em Python demonstrou robustez e aderência estrita aos modelos teóricos matemáticos estudados. A implementação de um motor determinístico baseado em dicionário de transições estáticas comprovou alta eficiência computacional no mapeamento de cadeias, garantindo um isolamento completo das camadas semânticas e de otimização de código intermediário.

O projeto permitiu consolidar na prática os desafios intrínsecos ao projeto de compiladores, como a correta manipulação de atributos sintetizados em árvores de derivação e o rastreamento linear de variáveis para eliminação de redundâncias em representações em três endereços. Como proposta de trabalhos futuros, sugere-se expandir o ecossistema sintático para suportar a recuperação dinâmica de erros utilizando estratégias de sincronização baseadas em tokens de descarte (*Panic Mode*).

Referências

- [1] Mello, B. (2026). *Descrição do PROJETO 2: Construção de Analisador Sintático*. UFFS - Campus Chapecó.
- [2] Aho, A. V., Lam, M. S., Sethi, R., e Ullman, J. D. (2008). *Compiladores: Princípios, Técnicas e Ferramentas*. 2ª edição. Addison-Wesley/Pearson.